Tecnológico Nacional De México

Instituto Tecnológico Superior De Guasave

Ingeniería En Sistemas Computacionales

Materia

Diseño de interfaz web

Alumno

Maldonado Valle Jesus Antonio (2125010015)

Grupo: 701

Fecha: 26/10/26

Mc: José esteban peral Huiza

Lugar: Guasave, Sinaloa, México

1. Configuración del Servidor (Node.js / Express)

En esta práctica se comparan dos estrategias de diseño web en Node.js con Express corriendo en el

puerto 3000:

• Ruta /responsive: Sirve un archivo HTML estático que se adapta en el cliente mediante CSS.

• Ruta /adaptive: Utiliza express-useragent para detectar el dispositivo en el servidor y renderizar

condicionalmente vistas EJS (desktop.ejs o mobile.ejs).

Código de detección en el servidor (server.js):

app.get('/adaptive', (req, res) => {

const isMobile = req.useragent.isMobile || req.headers['user-

agent'].includes('Mobile');

if (isMobile) {

res.render('mobile', { products: getMockProducts() });

} else {

res.render('desktop', { products: getMockProducts(), showChart: true });

}

});

2. Fase 2: Implementación Versión A (RWD - Responsive)

En la ruta /responsive, el servidor envía un único archivo HTML para todos los clientes. La

reestructuración visual depende del navegador del usuario a través de reglas de CSS Media Queries

(@media (max-width: 768px)).

Regla CSS clave (styles.css):

@media (max-width: 768px) {

.container { grid-template-columns: 1fr; }

.product-grid { grid-template-columns: 1fr; }

}

3. Fase 3: Implementación Versión B (AWD - Adaptativo)

En la ruta /adaptive, la lógica se ejecuta en el servidor. El middleware evalúa la cabecera User-Agent de

la petición HTTP recibida para renderizar desktop.ejs o mobile.ejs.

• En versión Escritorio: Se renderiza la interfaz completa incluyendo módulos de filtros avanzados y

gráficos de rendimiento pesados.

• En versión Móvil: Se genera un HTML limpio sin gráficos ni elementos pesados en el DOM, reduciendo

el consumo de procesamiento en celulares.

4. Fase 4: Auditoría y Medición de Rendimiento

Comparativa de métricas obtenidas desde las herramientas de desarrollo de Chrome al simular un

cliente móvil en ambas versiones:

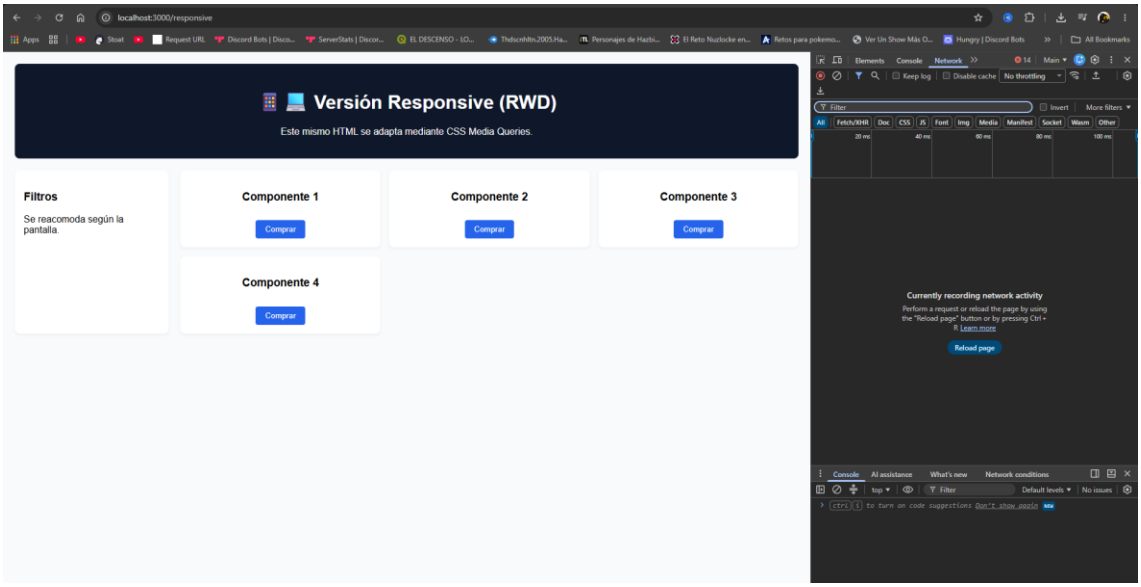
Métrica / Indicador	RWD (Móvil)	AWD (Móvil)	Delta (%)
Tamaño del documento inicial (HTML transferido)	~4.2 KB	~1.4 KB	-66.6%
Peso total de la página (Payload completo)	~18.5 KB	~6.2 KB	-66.4%
Número de peticiones HTTP (Requests)	4 reqs	2 reqs	-50.0%

Largest Contentful Paint (LCP)	1.2 s	0.5 s	-58.3%
Cumulative Layout Shift (CLS)	0.02	0.00	N/A

5. Evidencias Visuales de Funcionamiento

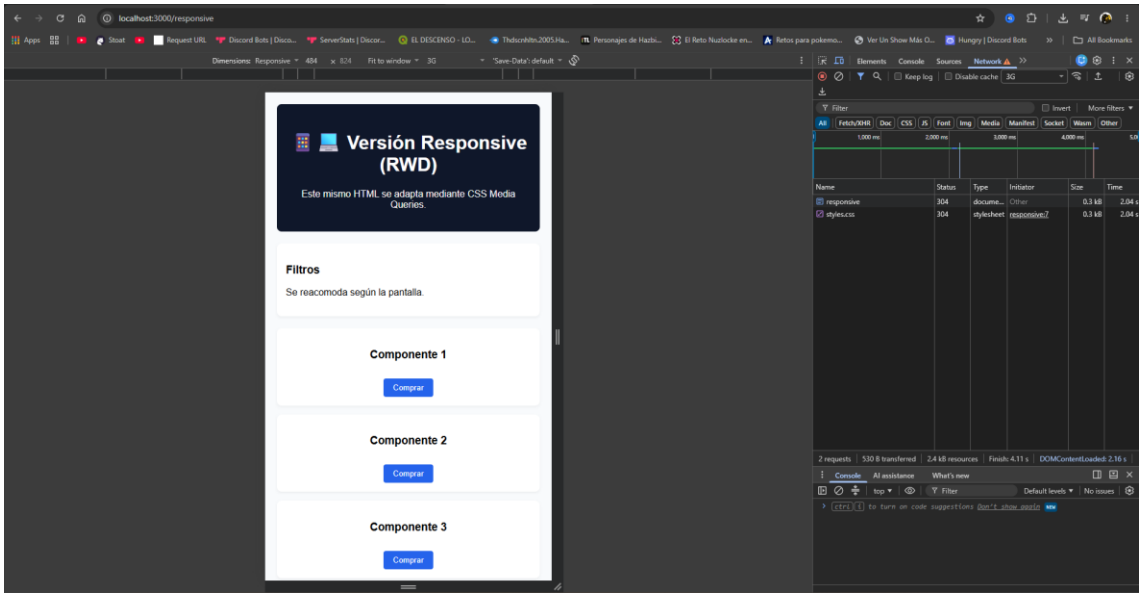
A continuación se presentan las capturas de pantalla que demuestran el funcionamiento de cada

estrategia en el navegador:



Explicación de la imagen: Refleja el funcionamiento de RWD en pantalla grande. Se observa la barra lateral fija a la

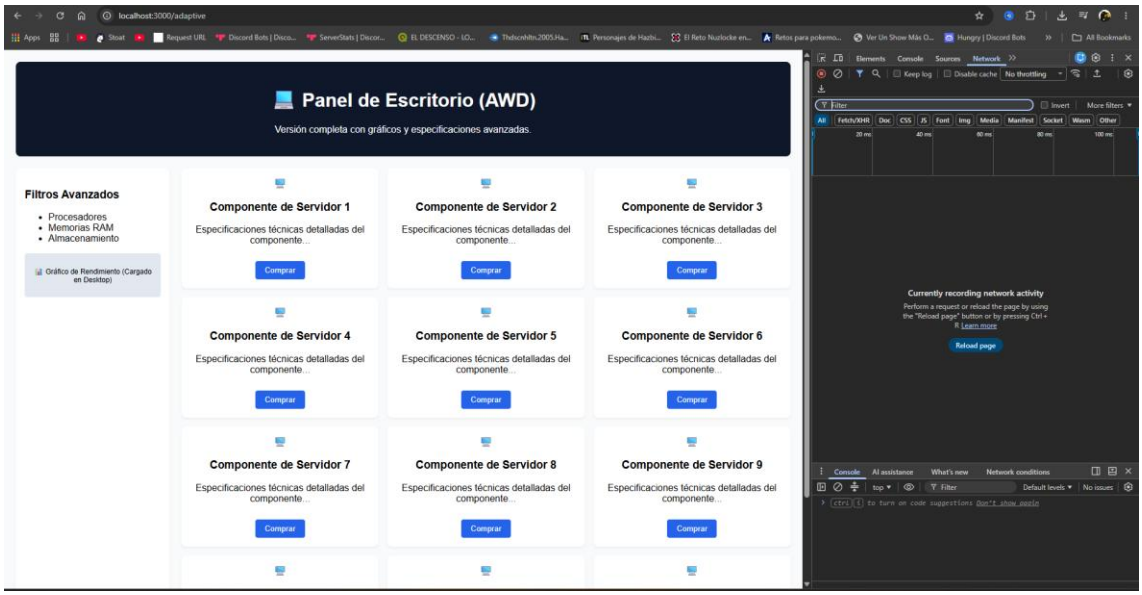
izquierda y la cuadrícula de componentes distribuida en 3 columnas.



Explicación de la imagen: Refleja el funcionamiento de RWD en móviles. Al simular una pantalla de 484px con

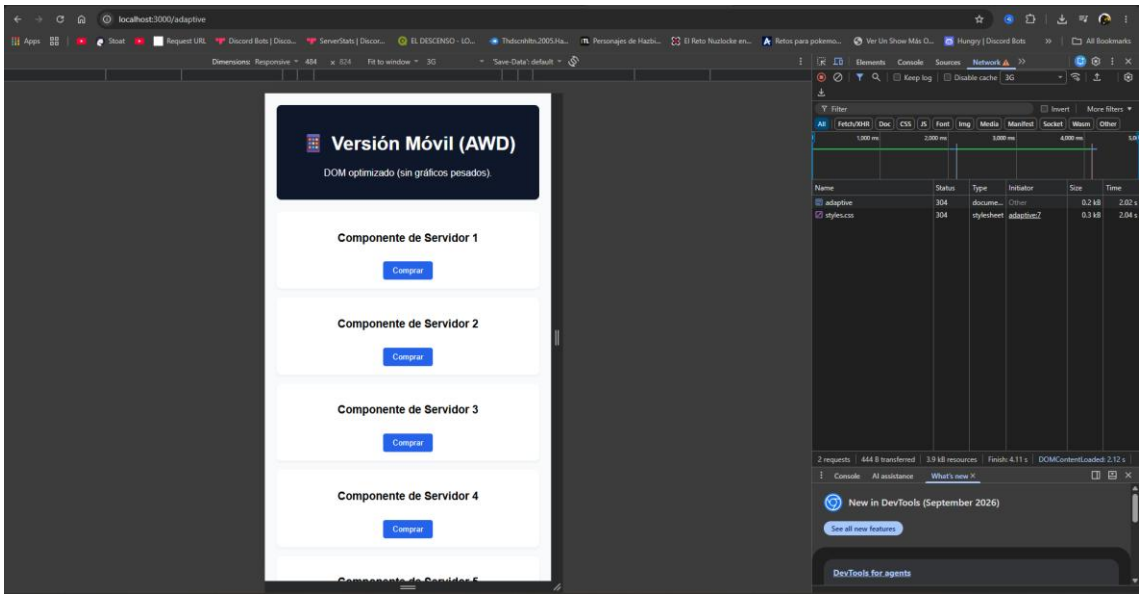
DevTools, las Media Queries de CSS colapsan la barra lateral a la parte superior y la cuadrícula pasa a 1 columna.

En el panel Network se confirma que el HTML se cargó una sola vez y solo la hoja de estilos aplica el cambio visual.



Explicación de la imagen: Refleja el funcionamiento de AWD en escritorio. Muestra la vista generada por

desktop.ejs con el panel lateral completo y el recuadro del gráfico de rendimiento.



Explicación de la imagen: Refleja el funcionamiento de AWD en móviles. Muestra la vista generada por mobile.ejs

donde el servidor envió un DOM limpio sin gráficos ni elementos pesados, optimizando la carga.

6. Conclusión

• RWD (Responsive): Mantiene un único código HTML/CSS, pero obliga a empaquetar y descargar la

estructura completa en todos los dispositivos.

• AWD (Adaptativo): Requiere más trabajo en el servidor, pero envía solo el código necesario para cada

dispositivo, mejorando la velocidad y el rendimiento en celulares.